

linked list

↳ each listNode/Node contain atleast two things

listNode	
value	next

→ Pointer to next Node

blue	red	green
xx	xy	xz

memory

Value

Address

listNode1	
red	—

listNode2	
blue	—

listNode3	
green	—

(Default)
null

listNode1.next = listNode2

listNode2.next = listNode3

Tips →

1) Always have variable pointing to the head and tail of a linked list for easy insertion.

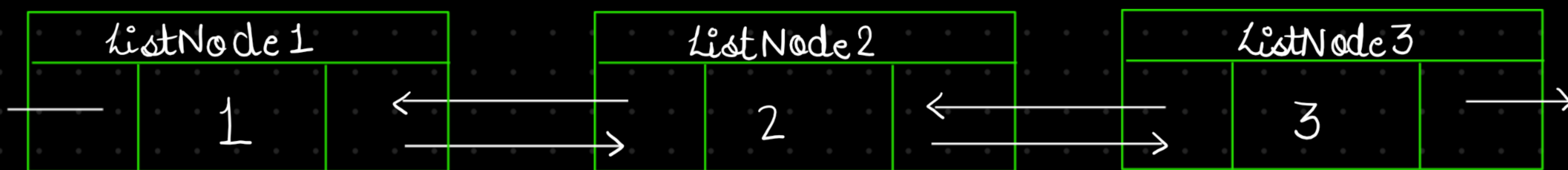
Doubly linked list

↳ each list contains three things.

listNode		
prev	val	next

↙ head

↘ tail



Implementation

1) Singly linked list

```
# include <iostream>
```

```
using std::cout
```

```
using std::endl;
```

```
class ListNode {
```

```
public:
```

```
    int val_;
```

```
    ListNode * = nullptr;
```

```
    ListNode(int val){
```

```
        val_ = val;
```

```
    }
```

```
};
```

```
class LinkedList {
```

```
public:
```

```
    ListNode * head;
```

```
    ListNode * tail;
```

```
    LinkedList(){
```

```
        // Init the list with a 'dummy' node which makes
```

```
        // removing a node from the beginning of list easier.
```

```
        head = new ListNode(-1);
```



```

    tail = head;
}

void insertEnd(int val){
    tail → next = new ListNode(val);
    tail = tail → next;
}

void remove(int index){
    int i = 0;
    ListNode * curr = head;
    while( i < index && curr ){
        i++;
        curr = curr → next;
    }
    // Remove the node ahead of curr
    if( curr && curr → next ){
        if( curr → next == tail ){
            tail = curr;
        }
        curr → next = curr → next → next;
    }
}

```



```
void print() {
```

```
    listNode * curr = head->next; // skipping dummy
```

```
    while(curr) {
```

```
        cout << curr->val << " -> ";
```

```
        curr = curr->next;
```

```
    }
```

```
    cout << endl;
```

```
}
```

```
};
```

Doubly Linked List

```
#include <iostream>
```

```
using std::cout;
```

```
using std::endl;
```

```
class LinkedList {
```

```
public:
```

```
    int val_;
```



```
class LinkedList {
```

```
public:
```

```
    ListNode * next;
```

```
    ListNode * tail;
```

```
    LinkedList() {
```

```
        head = new ListNode(-1);
```

```
        tail = new ListNode(-1);
```

```
    }
```

```
    void insertFront(int val) {
```

```
        ListNode * newNode = new ListNode(val);
```

```
        newNode->prev = head;
```

```
        newNode->next = head->next;
```

```
        head->next->prev = newNode;
```

```
        head->next = newNode;
```

```
    }
```

```
    void insertEnd(int val) {
```

```
        ListNode * newNode = new
```